



Le génie pour l'industrie

GUIDE UTILISATEUR

PRÉSENTÉ À L'ÉCOLE DE TECHNOLOGIE SUPÉRIEURE
DANS LE CADRE DU COURS
LOG795 PROJET DE FIN D'ÉTUDES EN GÉNIE LOGICIEL

DÉVELOPPEMENT D'UN OUTIL D'ÉTIQUETAGE AUTOMATIQUE DES PULL
REQUESTS VIA LES GRANDS MODÈLES DE LANGAGE (LLMS)

FAROUK MEBARKI MEBF71040201

TALIP KOYLUOGLU KOYT85110004

CHAWKI BENZEGHIBA BENC63050101

MEHDI YASSIN YASM24019802

MIA BEN REDJEB BENM69270201

DÉPARTEMENT DE GÉNIE LOGICIEL ET DES TI

Professeur-superviseur

ALI OUNI

MONTREAL, 10 AOÛT 2026
ÉTÉ 2026

Table des matières

1. Introduction.....	3
1.1 Présentation de PRIntentLabeler	3
1.2 Fonctionnement général	3
2. Prérequis et installation	4
2.1 Prérequis	4
2.2 Installer la GitHub App	4
3. Configuration du LLM	5
3.1 Connexion et sélection de l'installation	5
3.2 Choisir le fournisseur et le modèle.....	5
3.3 Entrer et tester la clé API.....	6
3.4 Enregistrer la configuration	7
3.5 Modifier, remplacer ou supprimer	7
4. Utilisation avec une Pull Request	8
4.1 Déclencher une analyse.....	8
4.4 Lire le commentaire d'analyse.....	9
5. Gestion des labels suggérés	11
6. Confidentialité et coûts	12
7. Dépannage.....	13
8. Aide-mémoire	15

1. Introduction

1.1 Présentation de PRIntentLabeler

PRIntentLabeler est une GitHub App conçue pour analyser automatiquement les Pull Requests et proposer des étiquettes qui représentent l'intention principale du changement. Une PR peut, par exemple, corriger un bogue, ajouter une fonctionnalité, mettre à jour une dépendance ou modifier uniquement la documentation. L'application aide le mainteneur à reconnaître cette intention plus rapidement, surtout lorsque la PR contient plusieurs fichiers ou une description très courte.

L'outil fonctionne directement dans GitHub. Il n'impose pas l'utilisation d'un tableau de bord séparé pour consulter les résultats. Après l'analyse, un commentaire est publié dans la PR avec les fichiers retenus, les labels suggérés, leur score de confiance et une justification. Une Check Run offre aussi des actions rapides pour appliquer les suggestions.

1.2 Fonctionnement général

- GitHub avertit l'application lorsqu'une PR est ouverte, synchronisée ou modifiée.
- Le backend récupère le titre, la description, les fichiers modifiés et les labels disponibles dans le dépôt.
- Les fichiers sont classés et filtrés afin de construire un contexte représentatif qui respecte le budget de jetons du modèle choisi.
- Le LLM propose au maximum trois labels déjà présents dans le dépôt.
- Le résultat est validé par le backend, puis publié ou mis à jour dans la PR.

À retenir Le score affiché à côté d'un fichier sert uniquement à prioriser le contexte. Il est calculé par le backend et n'est pas envoyé au LLM. Le pourcentage affiché à côté d'un label représente plutôt la confiance déclarée par le modèle.

1.3 À qui s'adresse ce guide ?

Ce guide vise principalement le mainteneur d'un dépôt ou le responsable qui installe l'application, configure le fournisseur LLM et consulte les résultats. Avant de commencer, il est recommandé de lire le fichier README.md du dépôt. La personne responsable d'un déploiement local devrait aussi consulter docs/SETUP_LOCAL.md; les détails de programmation seront présentés dans le guide programmeur.

2. Prerequisites et installation

2.1 Prerequisites

Avant de commencer, l'utilisateur doit posséder un compte GitHub et avoir le droit d'installer une GitHub App sur le compte ou l'organisation visée. Dans une organisation, une approbation peut être exigée par un propriétaire. Pour un premier essai, il est préférable d'utiliser un dépôt de test sur lequel il est possible de créer librement des branches et des Pull Requests.

Une clé API d'un fournisseur LLM est également requise, sauf si l'administrateur du serveur a configuré un fournisseur Groq de repli. La clé demeure rattachée au compte du fournisseur : ses quotas, ses limites et sa facturation s'appliquent donc aux analyses réalisées par PRIntentLabeler.

2.2 Installer la GitHub App

- Ouvrir le lien d'installation transmis par le responsable de PRIntentLabeler.
- Choisir le compte personnel ou l'organisation où l'application doit être installée.
- Accorder l'accès à tous les dépôts ou sélectionner uniquement les dépôts nécessaires.
- Vérifier les permissions demandées, puis confirmer l'installation.
- Attendre la redirection vers la page de configuration et se connecter avec GitHub.

2.3 Permissions utilisées

Permission	Utilisation
Metadata - lecture	Identifier le dépôt et obtenir ses informations de base.
Pull requests - lecture	Lire le titre, la description, les branches et les fichiers modifiés.
Issues - écriture	Publier le commentaire et appliquer ou retirer les labels d'une PR.
Checks - écriture	Créer la Check Run et présenter les actions rapides.

Conseil Pour un essai, accorder l'accès à un seul dépôt. Les dépôts autorisés pourront être modifiés plus tard depuis les paramètres d'installation de GitHub.

3. Configuration du LLM

3.1 Connexion et sélection de l'installation

Après l'installation, GitHub redirige normalement vers l'adresse /setup de l'application. PRIntentLabeler démarre alors une autorisation OAuth. Cette connexion sert uniquement à vérifier l'identité de l'utilisateur et à obtenir la liste des installations GitHub auxquelles il a accès. Le jeton utilisateur reçu pendant cette vérification n'est pas conservé dans la base de données.

Une fois connecté, la page /settings présente les numéros des installations accessibles. Il faut sélectionner l'installation qui correspond au compte ou à l'organisation désirée. La configuration enregistrée s'appliquera à tous les dépôts autorisés dans cette même installation.

3.2 Choisir le fournisseur et le modèle

Option	Quand l'utiliser
Groq	Pour utiliser les modèles offerts par Groq.
OpenAI	Pour appeler directement un modèle OpenAI.
Anthropic	Pour utiliser un modèle Claude.
Google Gemini	Pour utiliser l'API Gemini.
xAI	Pour utiliser un modèle Grok.
Perplexity	Pour utiliser l'API Sonar.
API compatible OpenAI	Pour un fournisseur, un proxy ou un serveur qui comprend le format OpenAI.

Le champ Modèle accepte un identifiant suggéré ou un identifiant saisi manuellement. Il faut utiliser le nom reconnu par l'API, qui n'est pas toujours identique au nom commercial affiché sur le site du fournisseur. Comme ces identifiants évoluent, la documentation du fournisseur demeure la référence en cas de doute.

Interface de la route /settings pour le choix du fournisseur et du modèle de LLM :

Configuration de PRIntentLabeler

Installation GitHub #139226599 — connecté en tant que **TalipKoyluoglu**

Le contenu des Pull Requests sélectionné par l'application sera envoyé au fournisseur choisi avec cette clé API.

Fournisseur

Anthropic (Claude) ▾

Modèle

claude-haiku-4-5-20251001

Un modèle suggéré peut être sélectionné ou un identifiant exact peut être saisi.

Nouvelle clé API

Laisser vide pour conserver la clé actuelle. La clé enregistrée ne peut pas être réaffichée.

URL de base personnalisée

https://llm.exemple.ca/v1

Requise seulement pour une API compatible OpenAI personnalisée.

Tester la connexion

Enregistrer

Changer d'installation

Configuration actuelle

Groq — llama-3.1-8b-instant — clé terminant par cSp4

Dernière modification : 2026-08-10 15 h 08 min 04 s

Supprimer la configuration

L'analyse LLM utilisera ensuite la configuration Groq du serveur, si elle existe.

Supprimer la clé et la configuration

3.3 Entrer et tester la clé API

La clé API doit provenir du fournisseur sélectionné. Lorsqu'on clique sur Tester la connexion, le backend crée temporairement l'adaptateur correspondant et effectue une petite requête. Ce test permet de détecter une clé invalide, un modèle inexistant, un quota épuisé ou un problème de facturation avant que la configuration soit utilisée sur une PR.

L'option API compatible OpenAI demande aussi une URL de base. Il peut s'agir, par exemple, d'un serveur privé ou d'une passerelle qui expose une API au même format que celle d'OpenAI. En production, cette URL doit utiliser HTTPS et ne doit contenir ni identifiant, ni paramètre de requête, ni ancre. Pour les fournisseurs déjà intégrés, l'URL est définie par l'application et n'a pas à être saisie.

3.4 Enregistrer la configuration

- Choisir le fournisseur et confirmer l'identifiant du modèle.
- Saisir la clé API et, seulement pour l'option personnalisée, l'URL de base.
- Cliquer sur Tester la connexion et vérifier le message de réussite.
- Cliquer sur Enregistrer. Le serveur teste encore la connexion avant de remplacer la configuration existante.

3.5 Modifier, remplacer ou supprimer

Pour modifier uniquement le modèle, il est possible de laisser le champ de clé vide : la clé enregistrée sera conservée. Une nouvelle clé devient obligatoire lorsqu'on change de fournisseur. Cette précaution empêche, par exemple, qu'une clé Gemini soit réutilisée par erreur avec OpenAI.

La page affiche seulement les quatre derniers caractères de la clé. La valeur complète ne peut pas être relue. Le bouton de suppression efface la configuration de l'installation; le serveur utilisera ensuite sa configuration Groq de repli si elle existe. Sans configuration enregistrée ni fournisseur de repli, les PR recevront un commentaire indiquant qu'aucun fournisseur LLM n'est disponible. La configuration demeure enregistrée après un redémarrage si PostgreSQL et la clé maîtresse de chiffrement sont conservés.

Validation Après l'enregistrement, ouvrir une PR de test et vérifier que le commentaire d'analyse ainsi que la Check Run LLM PR Labeler apparaissent. Si l'un de ces résultats manque, consulter la section 7.

4. Utilisation avec une Pull Request

4.1 Déclencher une analyse

L'analyse démarre automatiquement lorsque l'application reçoit l'un des événements suivants : ouverture d'une Pull Request, ajout d'un nouveau commit sur sa branche ou modification de son titre ou de sa description. Il n'est pas nécessaire de mentionner le bot ni d'ajouter une commande dans la conversation de la PR.

Le serveur doit toutefois être accessible au moment où GitHub transmet le webhook. Dans un environnement local exposé avec Smee.io, l'ordinateur du développeur, PostgreSQL et l'application doivent être en marche. Dans un environnement déployé, cette responsabilité revient à l'administrateur du service.

4.2 Ce que fait l'application

- Lire les métadonnées de la PR, les fichiers modifiés et les labels du dépôt.
- Attribuer un rôle à chaque fichier : source, test, documentation, dépendance, CI/CD, configuration ou autre.
- Classer les fichiers, exclure le contenu généré et sélectionner des diffs représentatives.
- Adapter le nombre de fichiers et la taille des extraits au budget de jetons du modèle configuré.
- Envoyer le contexte préparé au fournisseur et valider sa réponse JSON.
- Publier ou mettre à jour le commentaire, puis créer la Check Run.

4.3 Pourquoi tous les fichiers ne sont-ils pas envoyés ?

Une PR peut contenir des fichiers générés, des images, des snapshots ou des lockfiles très volumineux. Ces fichiers donnent parfois une indication utile sur la nature du changement, mais leur diff apporte peu d'information au modèle. PRIntentLabeler les conserve dans le résumé général tout en omettant leur contenu. La sélection cherche aussi à varier les rôles et les répertoires afin qu'une longue série de tests ne masque pas le fichier source principal.

Bon à savoir Le nombre maximal de fichiers est un plafond de sécurité. Le nombre réellement envoyé varie selon la taille de la PR et la fenêtre de contexte du modèle.

4.4 Lire le commentaire d'analyse

Le commentaire commence par un résumé des informations de la PR : numéro, titre, auteur, branches, nombre de fichiers et volume des changements. Il indique aussi combien de fichiers ont réellement fourni une diff au LLM et combien ont été résumés sans contenu.

llm-pr-labeler Bot commented 2 weeks ago • edited by TalipKoyluoglu

LLM PR Labeler — Analyse

L'application a bien reçu et analysé cette Pull Request.

Champ	Valeur
PR	#12
Titre	test(catalogue): ajouter le calcul de prix taxes incluses version Groq
Auteur	R12F1
Branche source	testllm
Branche cible	main
Changements	+35 / -1
Fichiers modifiés	2
Fichiers analysés par le LLM	2
Fichiers résumés seulement (lockfile/snapshot/généré)	0

Fichiers sélectionnés pour l'analyse

Les fichiers ci-dessous ont été sélectionnés automatiquement par le backend et utilisés pour construire le contexte envoyé au LLM. Le score est une priorité interne de tri (diagnostic), il n'est jamais envoyé au LLM.

Fichier	Statut	Changements	Score	Raisons
<code>service-catalogue/produits/domain/value_objects.py</code>	modified	+10/-0	19 pts	source role, matches PR title, small focused diff, diff available
<code>service-catalogue/produits/tests.py</code>	modified	+25/-1	15 pts	test role, matches PR title, substantial diff, diff available

Labels suggérés — coche ceux à appliquer

Coche/décoche une case pour appliquer ou retirer le label correspondant sur cette PR.
Un label appliqué par l'IA porte toujours l'icône  devant son nom (ex.  bug) pour le distinguer d'un label ajouté manuellement par un humain, qui n'affiche jamais cette icône.

☐  enhancement — 92% — Added new method `appliquer_taxe` to `PrixMonetaire` class and tests for it

Aucun label appliqué pour l'instant.

Résumé

Added new method `appliquer_taxe` to `PrixMonetaire` class and tests for it.

Fichiers sélectionnés

Le tableau présente le nom du fichier, son statut, le nombre d'ajouts et de suppressions, son score de priorité ainsi que les raisons principales du classement. Ce score aide à diagnostiquer la sélection. Il ne mesure pas la qualité du fichier et n'est jamais présenté au LLM comme une preuve.

Labels suggérés

Chaque suggestion contient le nom exact d'un label du dépôt, un pourcentage de confiance et une courte justification. Le backend retire les labels inexistants, les doublons et les suggestions sous le seuil minimal de 70 %. Il conserve au plus trois résultats, classés par confiance décroissante.

Résumé

Lorsque le modèle fournit un résumé valide, celui-ci décrit en une phrase le changement principal observé. Il sert d'aide à la lecture et ne remplace pas la revue du code. Le score de confiance demeure lui aussi une estimation du LLM, et non une probabilité mathématique parfaitement calibrée.

4.5 Absence de suggestion ou échec

Un commentaire sans label ne signifie pas toujours que le modèle est en panne. Il est possible qu'aucun label du dépôt n'atteigne le seuil, que le dépôt ne possède aucun label pertinent ou que le fournisseur ne soit pas configuré. En cas d'erreur, le commentaire précise généralement si la clé a été refusée, si le modèle est introuvable, si le quota est atteint ou si la réponse n'a pas respecté le format attendu.

Après un nouveau commit ou une modification de la description, une nouvelle analyse remplace le contenu du commentaire existant. L'application utilise un marqueur invisible pour retrouver son propre commentaire, ce qui évite d'en ajouter un nouveau à chaque exécution.

Limites à garder en tête L'application conserve au plus trois labels et choisit uniquement parmi ceux qui existent déjà dans le dépôt. Certains fichiers ou diffs peuvent être omis pour respecter le budget de jetons. La confiance affichée reste une indication du LLM : le mainteneur doit toujours vérifier la suggestion avant de la conserver.

5. Gestion des labels suggérés

5.1 Utiliser les cases à cocher

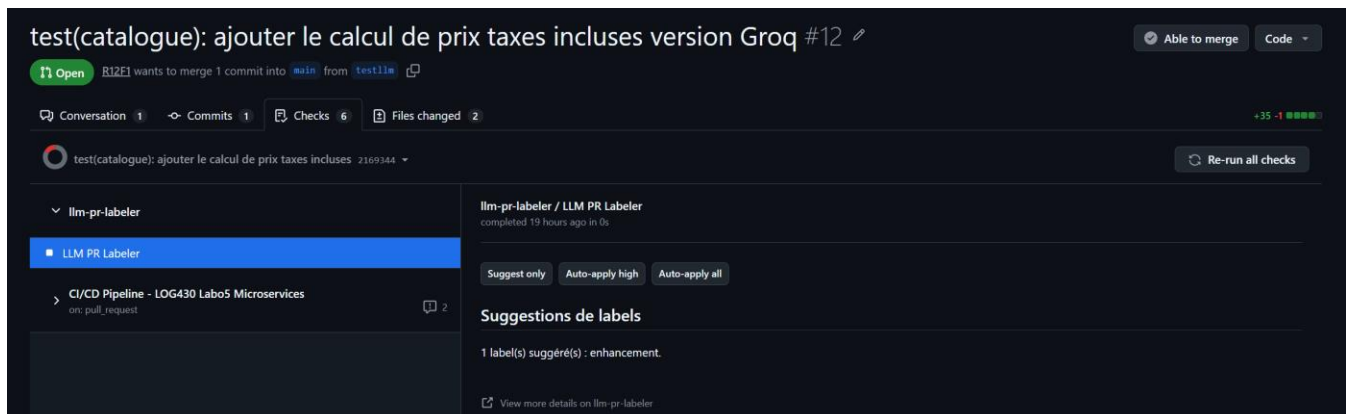
Les suggestions sont affichées comme des cases à cocher dans le commentaire de PRIntentLabeler. Pour appliquer ou retirer un label, le mainteneur édite le commentaire, coche ou décoche la case, puis enregistre la modification. Par sécurité, l'application traite seulement son commentaire officiel, une analyse signée et les labels réellement suggérés.

5.2 Comprendre le préfixe robot

Lorsqu'un label est appliqué par l'application, une icône de robot est placée devant son nom. Cette variante permet de distinguer une décision de PRIntentLabeler d'un label ajouté manuellement. La couleur du label original est réutilisée et l'application ne retire jamais sa variante humaine sans préfixe.

5.3 Actions de la Check Run

Dans la Pull Request, ouvrir l'onglet Checks, puis choisir LLM PR Labeler. Si la liste des contrôles est réduite, cliquer d'abord sur Show all checks ou sur Details. Les trois actions s'affichent dans la page de résultat de la Check Run.



Action	Effet sur l'analyse courante
Suggest only	Conserve les suggestions sans appliquer automatiquement de nouveaux labels.
Auto-apply high	Applique les suggestions dont la confiance atteint le seuil élevé de 85 %.
Auto-apply all	Applique toutes les suggestions validées, jusqu'à un maximum de trois.

Ces boutons agissent sur l'analyse publiée sans relancer le LLM. Le réglage LABEL_MODE définit séparément le comportement par défaut du serveur : aucune application en mode suggest, seulement les résultats très confiants en auto-high, et toutes les suggestions validées en auto-all.

5.4 Analyse périmée

Une action est refusée si un nouveau commit a été ajouté depuis l'analyse. Le SHA enregistré dans le commentaire doit correspondre au dernier commit de la PR et à celui de la Check Run. Il suffit alors d'attendre la nouvelle analyse ou de modifier la description de la PR pour en déclencher une autre.

6. Confidentialité et coûts

6.1 Données transmises au fournisseur

Pour classifier une PR, PRIntentLabeler transmet au fournisseur choisi le titre, une version limitée de la description, les statistiques de changement, un résumé des fichiers, les diffs sélectionnés et une liste réduite des labels disponibles. Le contenu complet de la PR n'est pas envoyé sans filtrage. Les fichiers générés, binaires, snapshots et lockfiles sont généralement résumés sans leur diff.

Le fournisseur reçoit uniquement le contexte nécessaire à cette analyse, mais ce contexte peut tout de même contenir du code source. Avant d'utiliser l'application sur un dépôt privé ou sensible, il faut vérifier les règles de l'organisation et les conditions du fournisseur. Le choix du fournisseur appartient à l'utilisateur qui configure l'installation.

6.2 Protection de la clé API

La clé est testée avant son enregistrement, puis chiffrée avec AES-256-GCM. PostgreSQL conserve le texte chiffré, un vecteur d'initialisation, une balise d'authentification et les quatre derniers caractères en clair. La clé maîtresse utilisée pour le déchiffrement reste dans l'environnement du serveur et n'est pas enregistrée dans la base.

Les opérations de création, de modification et de suppression sont inscrites dans une table d'audit sans y placer la clé API. Le jeton OAuth de l'utilisateur n'est pas sauvegardé; l'interface utilise plutôt une session aléatoire d'une durée limitée.

6.3 Quotas et facturation

PRIntentLabeler suit une approche Bring Your Own Key. Les requêtes sont donc comptabilisées sur le compte du fournisseur associé à la clé. Les quotas gratuits, les limites de fréquence, les crédits et la facturation sont déterminés par ce fournisseur. L'application réduit le contexte pour limiter l'utilisation de jetons, sans pouvoir garantir un coût fixe pour toutes les PR.

Bonnes pratiques Utiliser une clé réservée à l'application, surveiller sa consommation, limiter son accès selon les options du fournisseur et la remplacer immédiatement si elle a été exposée. Ne jamais placer une clé dans une PR, un commentaire GitHub ou un fichier versionné.

7. Dépannage

7.1 L'installation n'apparaît pas dans /settings

Vérifier d'abord que la GitHub App est installée sur le bon compte ou la bonne organisation. L'utilisateur connecté doit avoir accès à cette installation. Dans une organisation, il peut être nécessaire d'obtenir l'approbation d'un propriétaire. Si les droits viennent d'être modifiés, se déconnecter de PRIntentLabeler, puis recommencer l'autorisation GitHub pour renouveler la liste des installations.

7.2 La page indique que la configuration serveur est incomplète

Ce message signifie généralement que PostgreSQL, OAuth GitHub ou la clé de chiffrement n'est pas disponible sur le serveur. L'utilisateur final ne peut pas corriger ces éléments depuis l'interface. Il doit transmettre le message au responsable du déploiement en précisant l'adresse visitée et l'heure approximative de l'erreur.

7.3 Le test de connexion échoue

Erreur 401 ou 403. La clé est invalide, expirée ou n'a pas accès au modèle. Vérifier qu'elle appartient au fournisseur sélectionné et qu'elle ne contient aucun espace ajouté lors de la copie.

Erreur 404. Le fournisseur ne reconnaît pas l'identifiant du modèle. Copier l'identifiant exact depuis sa documentation ou choisir un autre modèle accessible au compte.

Erreur 429. Le quota ou la limite de requêtes est atteint. Attendre sa réinitialisation, choisir un modèle moins limité ou ajuster la facturation chez le fournisseur.

Erreur 500, 502, 503 ou 504. Le service est temporairement indisponible. L'application tente automatiquement plusieurs appels avant d'abandonner; réessayer un peu plus tard.

7.4 L'URL personnalisée est refusée

L'URL est acceptée uniquement pour l'option API compatible OpenAI. En production, elle doit utiliser HTTPS. Elle ne peut pas contenir un nom d'utilisateur, un mot de passe, des paramètres après un point d'interrogation ou une ancre après le symbole #. Une adresse locale comme localhost est refusée en production.

7.5 Aucun commentaire n'apparaît sur la PR

Confirmer que l'application est bien autorisée sur le dépôt et que la PR a été ouverte ou modifiée après l'installation. Dans un environnement local, vérifier que le serveur est démarré et qu'il reçoit les webhooks. Dans un environnement déployé, le responsable peut consulter les journaux du backend pour savoir si GitHub a transmis l'événement et si la lecture de la PR a échoué.

Une permission manquante peut empêcher la publication. L'application a besoin de la lecture des Pull Requests et de l'écriture sur les Issues pour créer son commentaire. La permission Checks est nécessaire pour la Check Run, mais son absence ne devrait pas empêcher le commentaire principal d'être publié.

7.6 Le commentaire ne contient aucun label

Vérifier que le dépôt possède des labels correspondant à une intention de changement. `PRIntentLabeler` ne crée pas librement une taxonomie pour le LLM : le modèle doit choisir un nom déjà disponible. Dans un dépôt sans label ou composé uniquement de labels de statut très spécialisés, aucune suggestion peut être retenue.

Le modèle peut aussi avoir produit des suggestions sous le seuil minimal de 70 %, une réponse invalide ou aucun texte exploitable. Le message affiché sous la liste des labels permet de distinguer ces cas d'un problème de configuration.

7.7 Un label ou une action ne fonctionne pas

Si une case cochée ne produit aucun changement, vérifier que le commentaire a bien été enregistré après l'édition. Seul le commentaire officiel créé par l'application est traité. Une copie du commentaire, même identique visuellement, est ignorée.

Si un bouton de la Check Run ne fait rien après un nouveau push, l'analyse est probablement périmée. Attendre que le commentaire soit mis à jour. Enfin, l'application ne modifiera pas un label humain sans préfixe robot; elle ajoute ou retire uniquement sa propre variante.

7.8 Obtenir de l'aide

Lorsqu'un problème persiste, noter le dépôt, le numéro de PR, l'action effectuée, le fournisseur et le message exact. Ne jamais joindre la clé API. Ces informations permettent au responsable de retrouver rapidement l'événement correspondant dans les journaux.

8. Aide-mémoire

8.1 Première configuration

- Installer la GitHub App sur un dépôt de test.
- Se connecter à /settings avec GitHub.
- Choisir l'installation à configurer.
- Sélectionner le fournisseur et le modèle.
- Entrer la clé API et tester la connexion.
- Enregistrer la configuration.
- Ouvrir une PR et vérifier le commentaire produit.

8.2 Utilisation courante

Après chaque ouverture, synchronisation ou modification d'une PR, attendre la mise à jour du commentaire. Examiner les fichiers sélectionnés, puis comparer les labels proposés avec les changements réels. Appliquer les suggestions avec les cases à cocher ou la Check Run seulement après cette vérification. Un score élevé facilite la décision, mais ne remplace pas le jugement du mainteneur.

8.3 Glossaire rapide

Terme	Définition
Pull Request (PR)	Proposition de modifications à intégrer dans une branche d'un dépôt GitHub.
Installation GitHub	Lien entre la GitHub App, un compte ou une organisation et les dépôts autorisés.
LLM	Grand modèle de langage utilisé pour interpréter le contexte de la PR.
Clé API	Secret permettant au backend d'appeler le fournisseur au nom de l'utilisateur.
Webhook	Événement envoyé par GitHub au serveur lorsqu'une action survient.
Check Run	Résultat visible dans l'onglet Checks et contenant les actions rapides de l'application.
Jeton	Unité de texte utilisée pour calculer la taille du contexte transmis au modèle.

Rappel final PRIntentLabeler est un outil d'aide à la classification. Le mainteneur demeure responsable de vérifier les suggestions avant de les conserver sur une Pull Request.